

Forecasting an Autoregressive Process

Course: Practical Time Series Analysis

Lecturer: Dr. David Zmiaikou

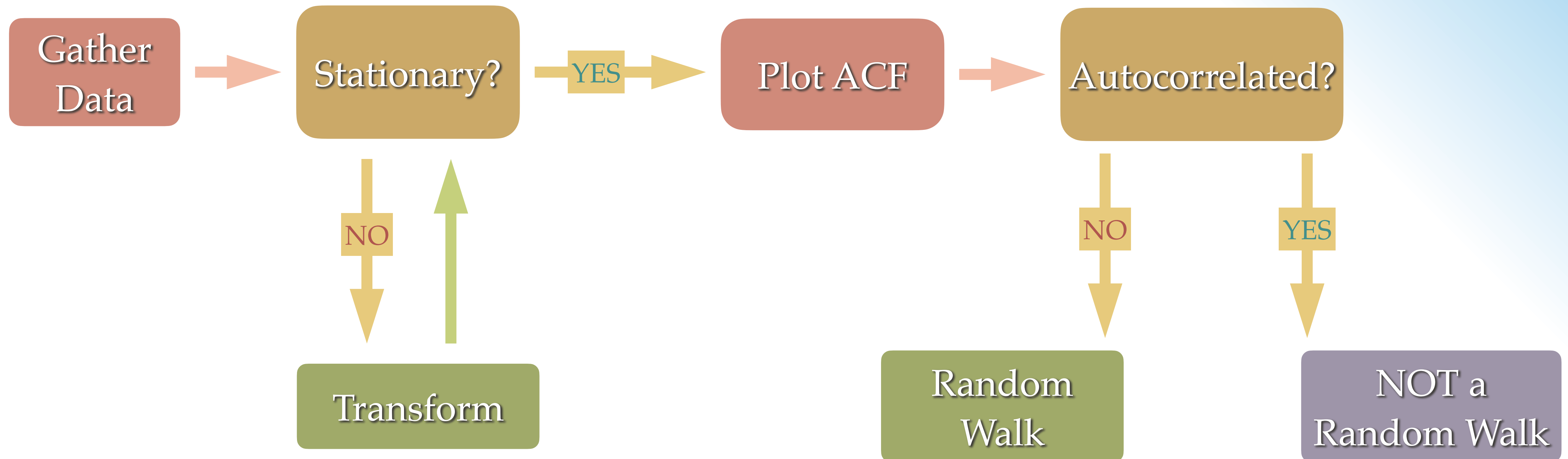
May 1, 2024

Contents

01 Reminders

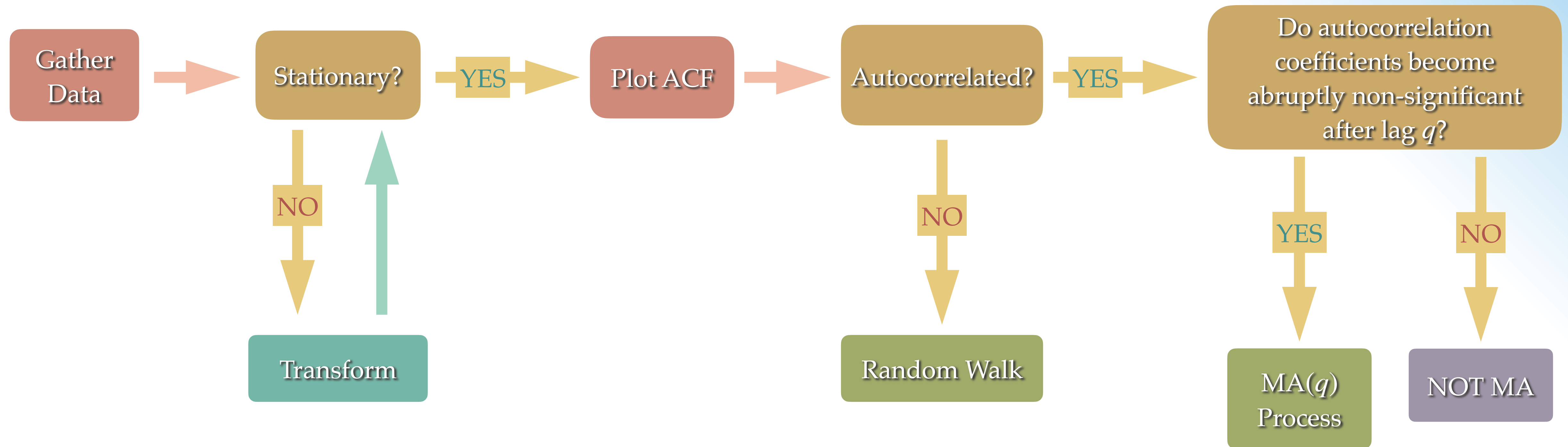
02 Forecasting an Autoregressive Process

Identifying a Random Walk

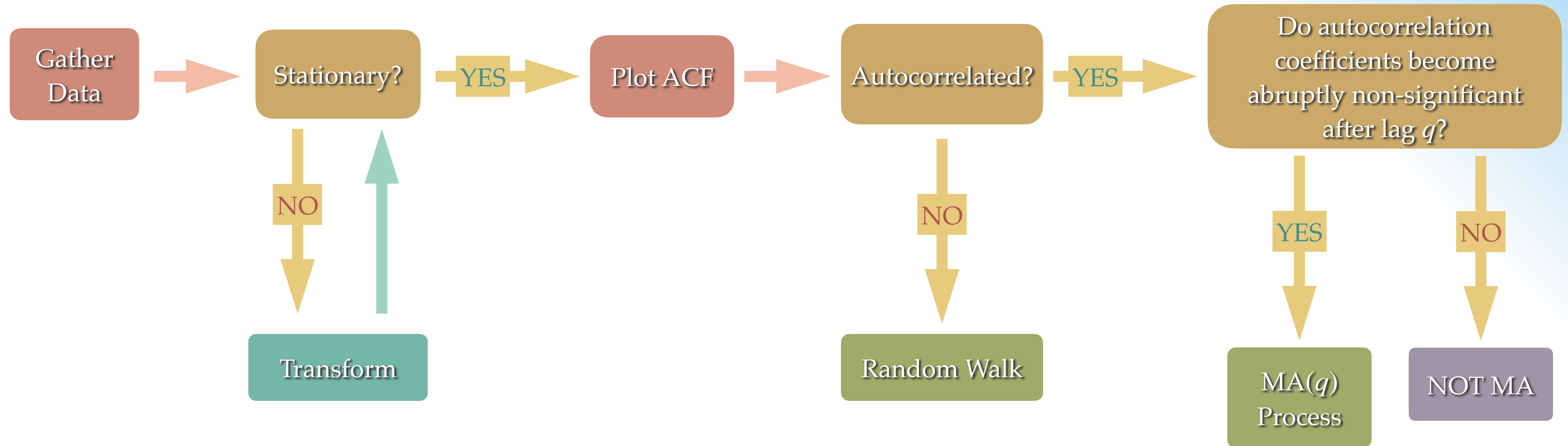


In the context of time series, **a random walk** is defined as a series whose first difference is *stationary* and *uncorrelated*.

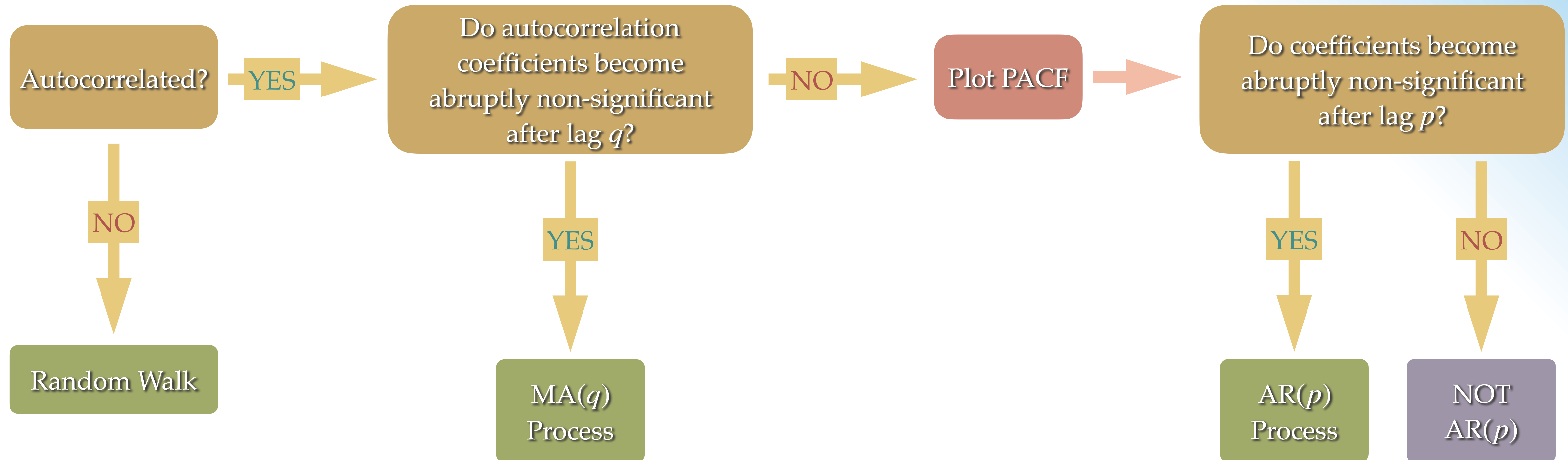
Identifying a Moving Average Process



Identifying an Autoregressive Process



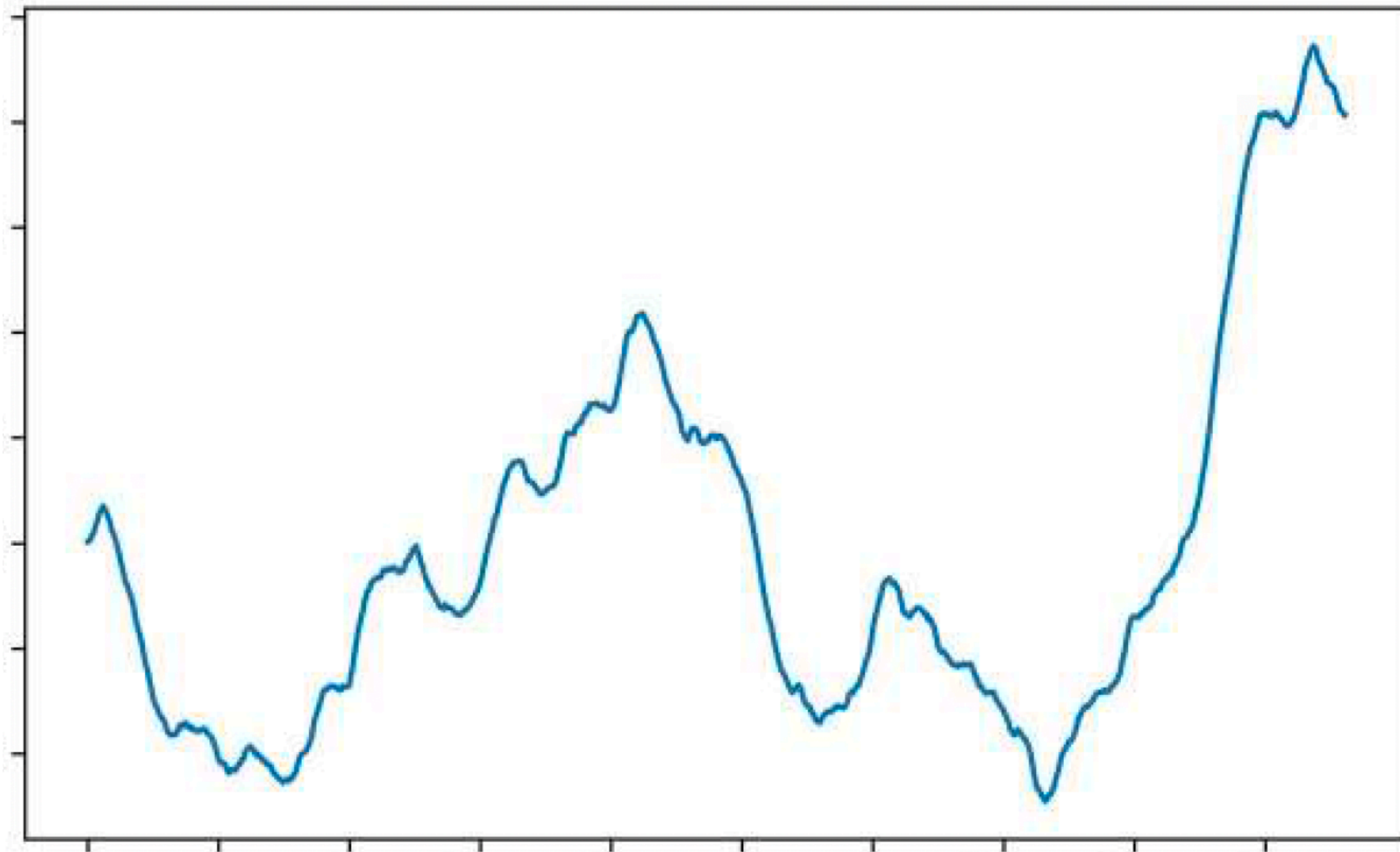
Identifying an Autoregressive Process



01

Forecasting an Autoregressive Process

^{def} What is an Autoregressive Process?

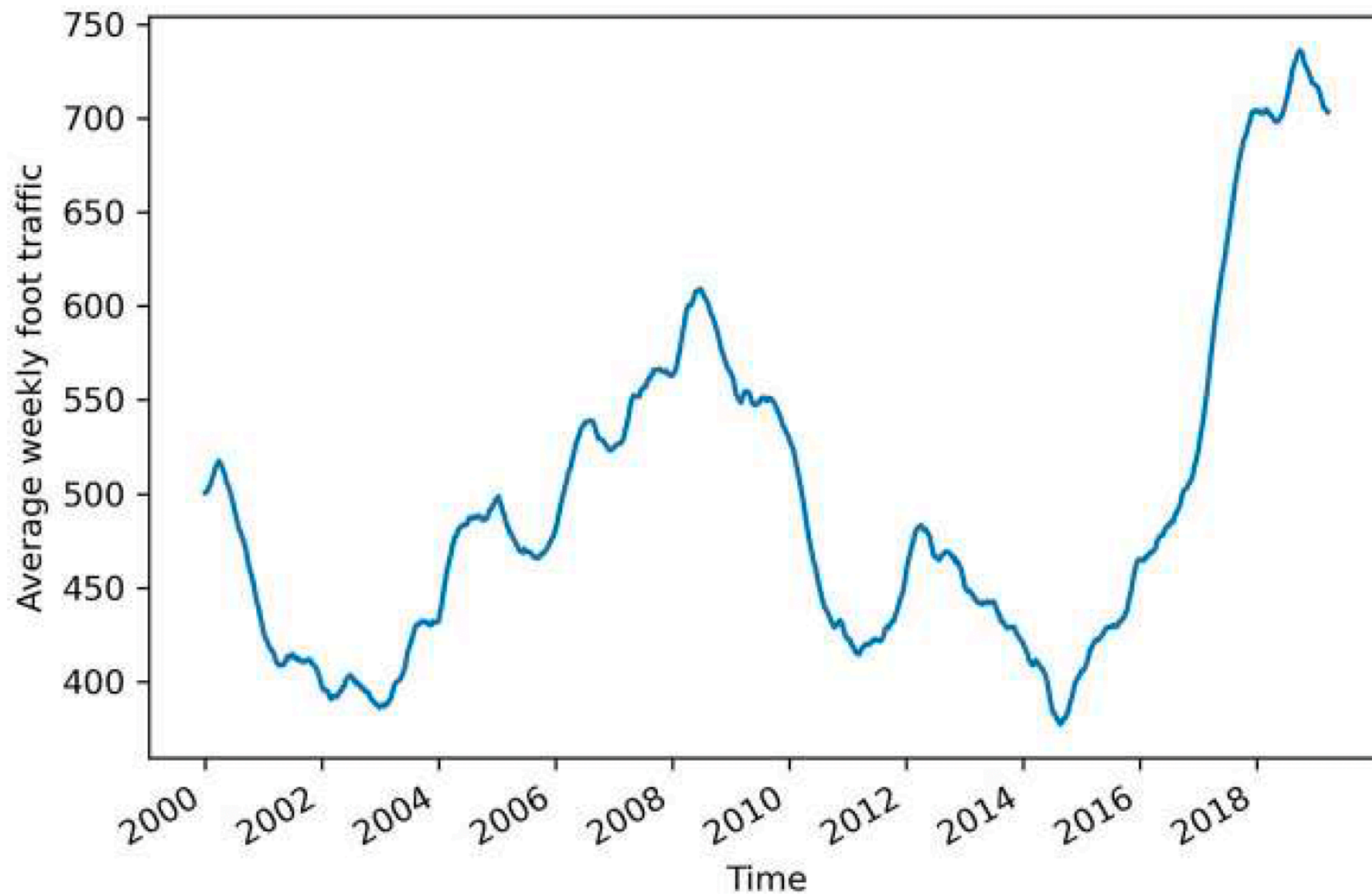


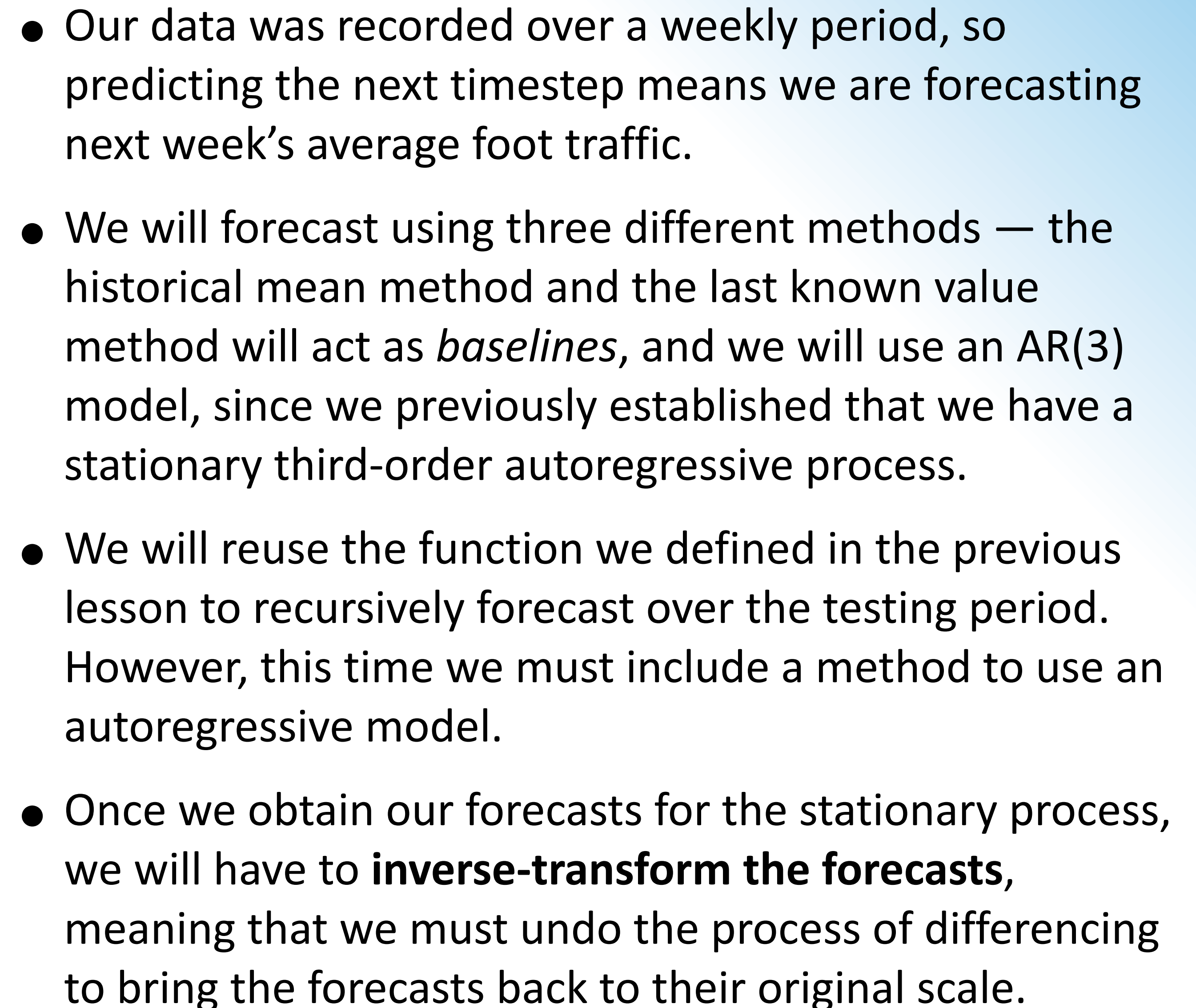
- An **autoregressive process** is a regression of a variable against itself. In a time series, this means that the present value is linearly dependent on its past values.
- The autoregressive process is denoted as $AR(p)$, where p is the **order**. The general expression of an $AR(p)$ model is

$$y_t = C + \epsilon_t + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \dots + \phi_p y_{t-p},$$

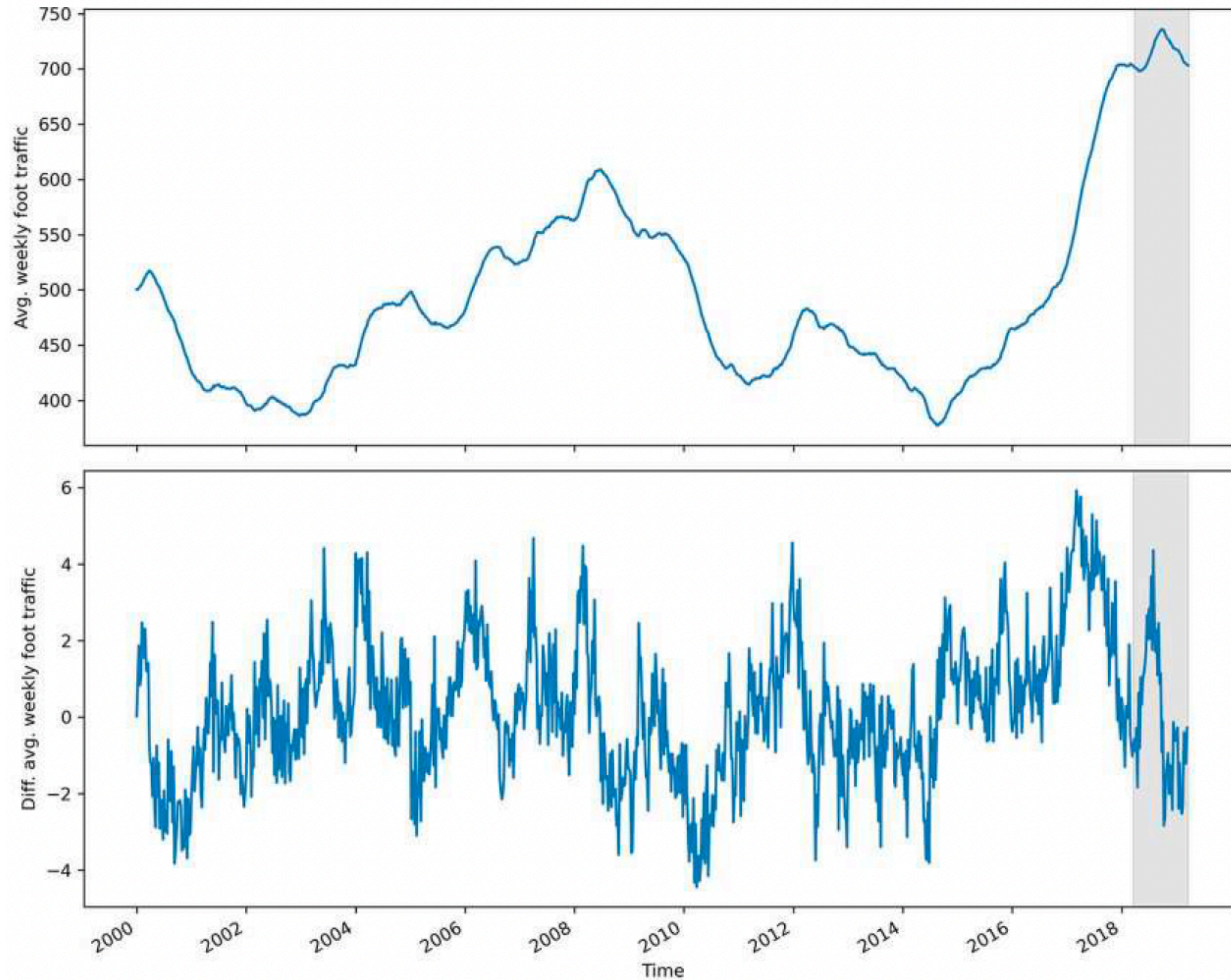
where C is a constant, ϵ_t is an error (white noise) and ϕ_t are coefficients (magnitude of influence of past values).

Example 1: Average weekly foot traffic at a retail store





Forecasting an Autoregressive Process



- In order to evaluate our forecasts, we will hold out the last 52 weeks of data for our test set, while the rest will be used for training. That way, we can evaluate the performance of our forecast over a period of 1 year:

```
import pandas as pd
```

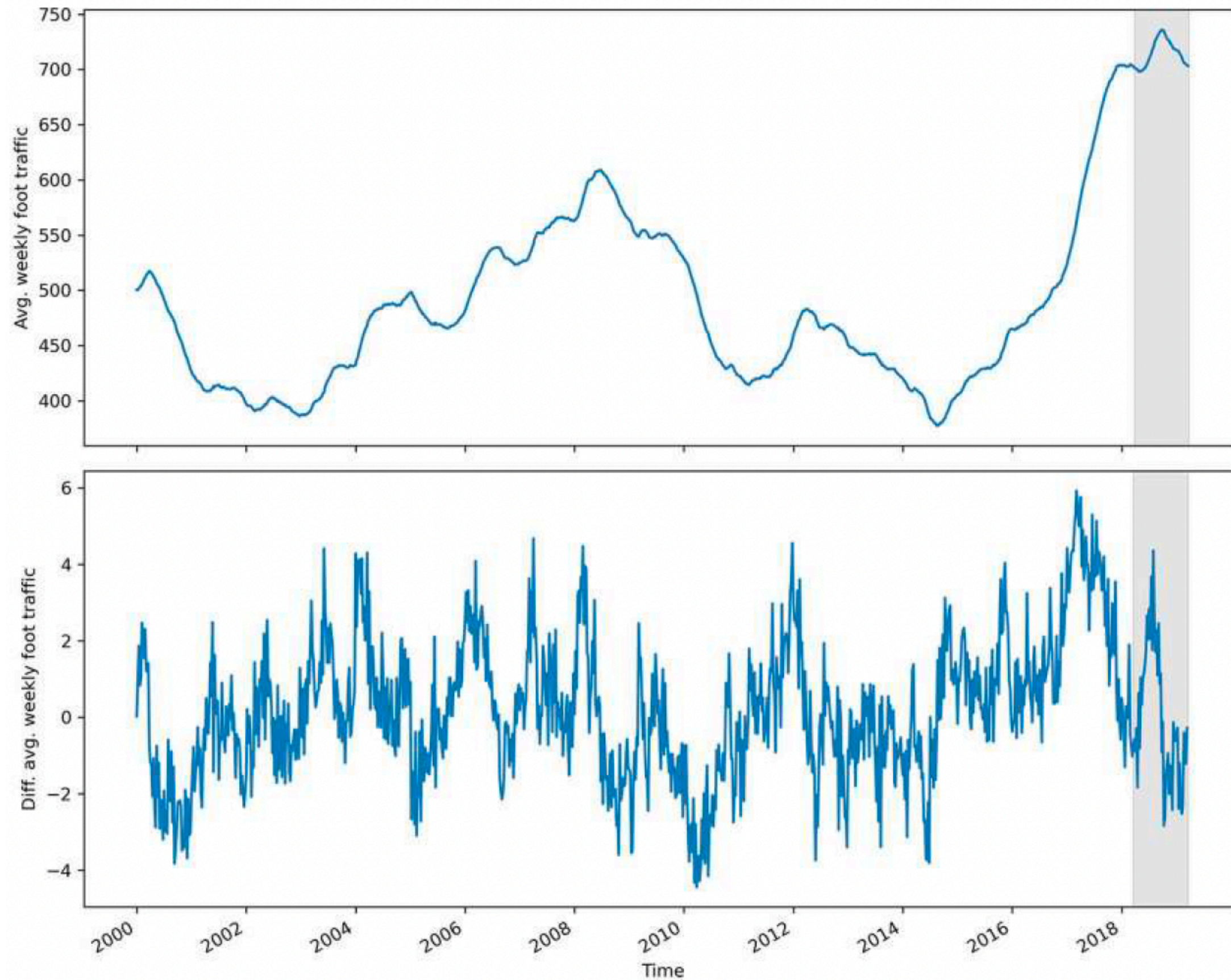
```
df_diff = pd.DataFrame({foot_traffic_diff':  
foot_traffic_diff})
```

```
train = df_diff[:-52]
```

```
test = df_diff[-52:]
```

- Note that the sum of both sets gives 999, which is one less data point than our original series.

Forecasting with the AR(p) Model



- We can use rolling forecasts:

```
def rolling_forecast(df: pd.DataFrame, train_len: int,  
horizon: int, window: int, method: str) -> list:
```

- Parameters:
 - **train_len** is the number of data points that can be used to fit a model;
 - **horizon** is the length of the test set and represents how many values must be predicted;
 - **window** specifies how many timesteps are predicted at a time;
 - **method** specifies what model to use.

Forecasting an Autoregressive Process:

Historical Mean

- Each forecasting method is run in a loop. The loop starts at the end of the training set and continues until `total_len`, exclusive, with steps of `window` (`total_len` is the sum of `train_len` and `horizon`):

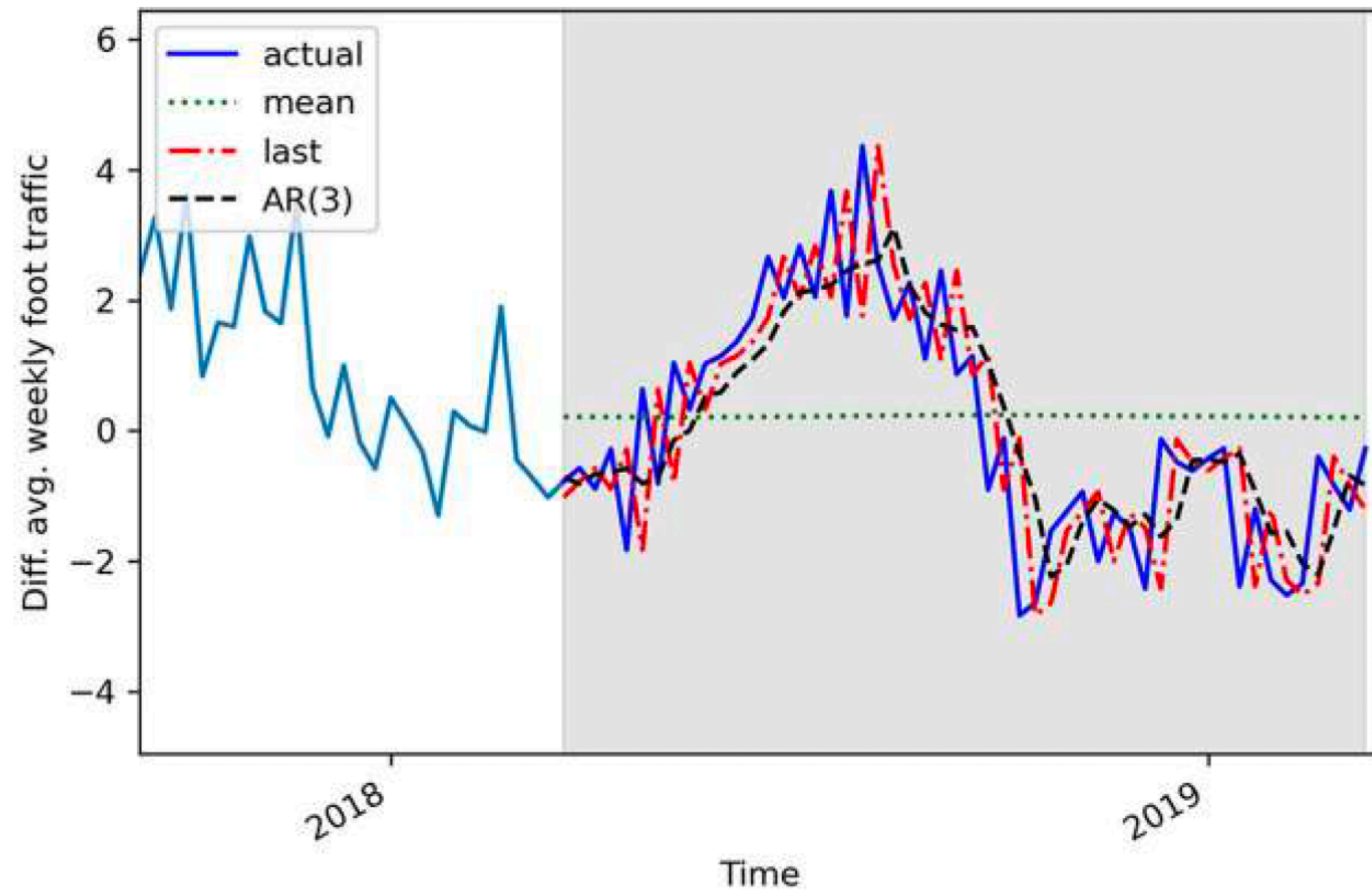
`total_len = train_len + horizon`

**`if method == 'mean':`
`pred_mean = []`**

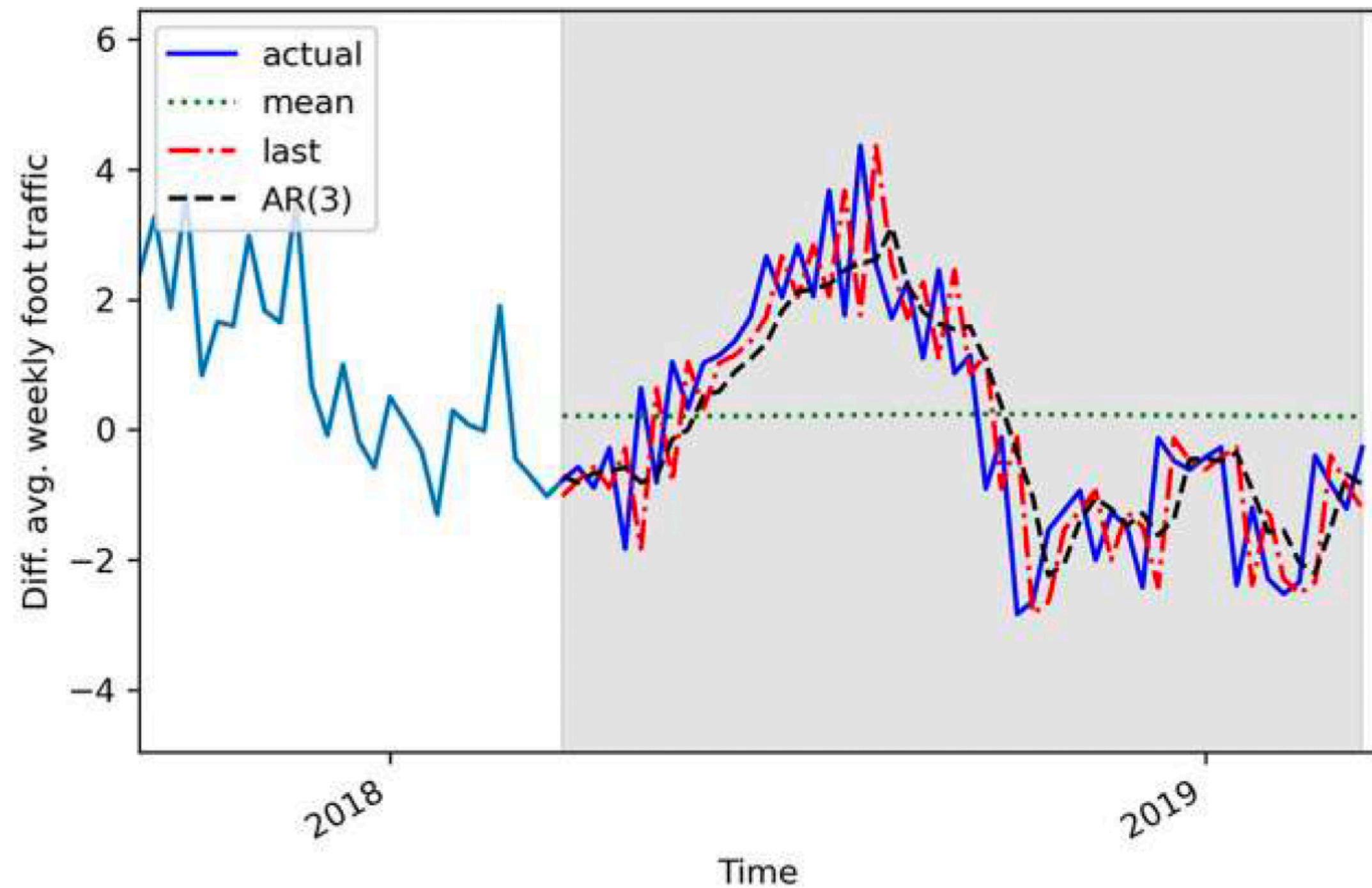
**`for i in range(train_len, total_len, window):`
`mean = np.mean(df[:i].values)`
`pred_mean.extend(mean for _ in range(window))`**

`return pred_mean`

- The loop generates a list of 52 forecasts for the entire test set.



Forecasting an Autoregressive Process: Last Known Value



- Another possible baseline is to predict the last known value of the train set:

```
elif method == 'last':  
    pred_last_value = []
```

```
for i in range(train_len, total_len, window):  
    last_value = df[:i].iloc[-1].values[0]  
    pred_last_value.extend(last_value for _  
                           in range(window))
```

```
return pred_last_value
```

Forecasting an Autoregressive Process: AR(3)

- Import the **SARIMAX** function from the **statsmodels** library:

```
from statsmodels.tsa.statespace.sarimax import SARIMAX
```

- This function fits an AR(3) model to our differenced series. Note that SARIMAX is a complex model that allows to consider seasonal effects, autoregressive processes, non-stationary time series, moving average processes, and exogenous variables all in a single model.

```
elif method == 'AR':
```

```
    pred_AR = []
```

```
    for i in range(train_len, total_len, window):
```

```
        model = SARIMAX(df[:i], order=(3,0,0))
```

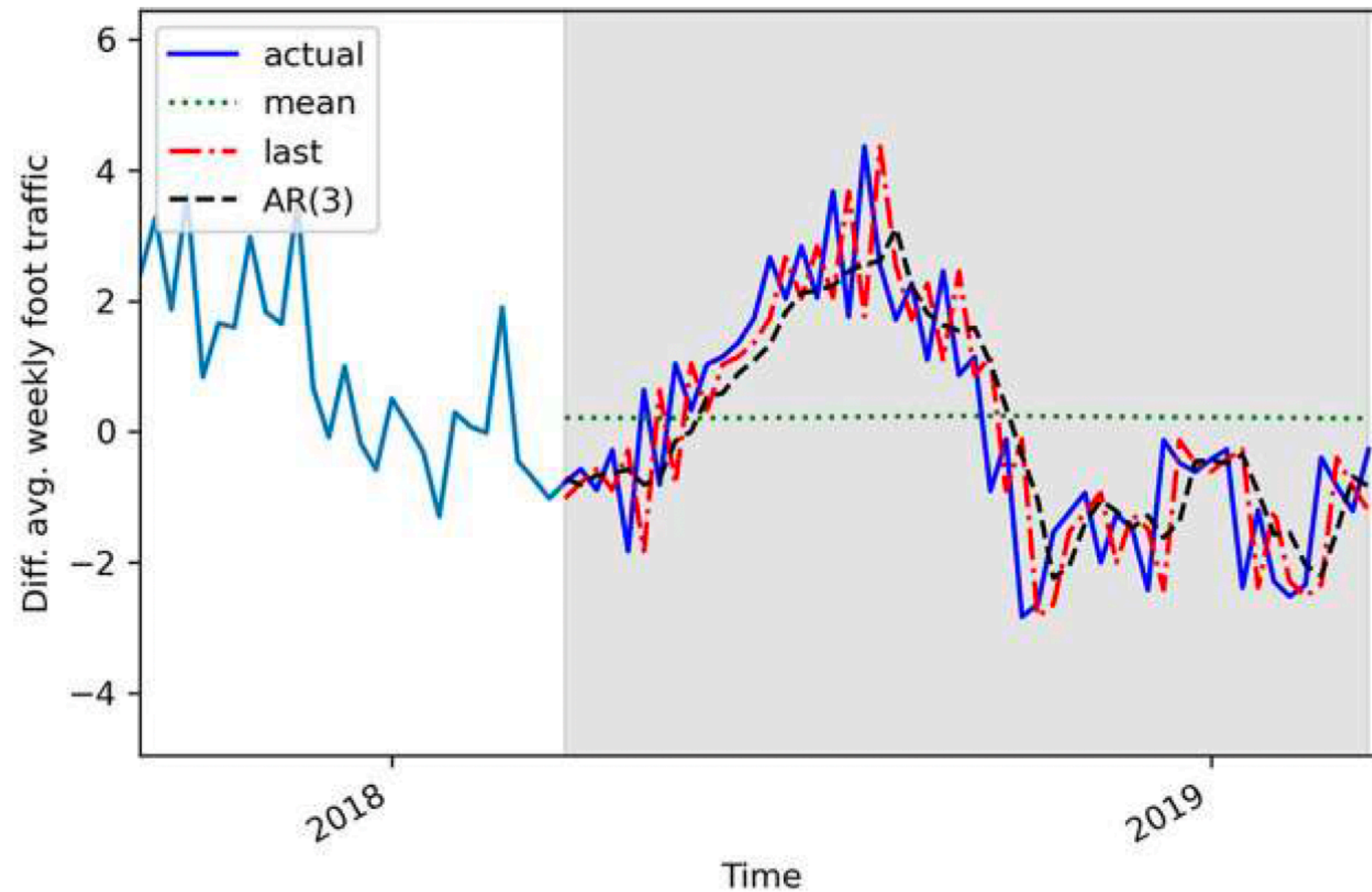
```
        res = model.fit(dis=False)
```

```
        predictions = res.get_prediction(0, i + window - 1)
```

```
        pred = predictions.predicted_mean.iloc[-window:]
```

```
        pred_AR.extend(pred)
```

```
    return pred_AR
```



Forecasting an Autoregressive Process

- Use our **rolling_forecast** function to generate a list of predictions for each method. Each list of predictions is then stored in its own column in **pred_df**:

```
pred_df = test.copy()
```

```
TRAIN_LEN = len(train)
```

```
HORIZON = len(test)
```

```
WINDOW = 1 # we wish to predict the next timestep
```

```
pred_mean = rolling_forecast(df_diff,  
                             TRAIN_LEN, HORIZON, WINDOW, 'mean')
```

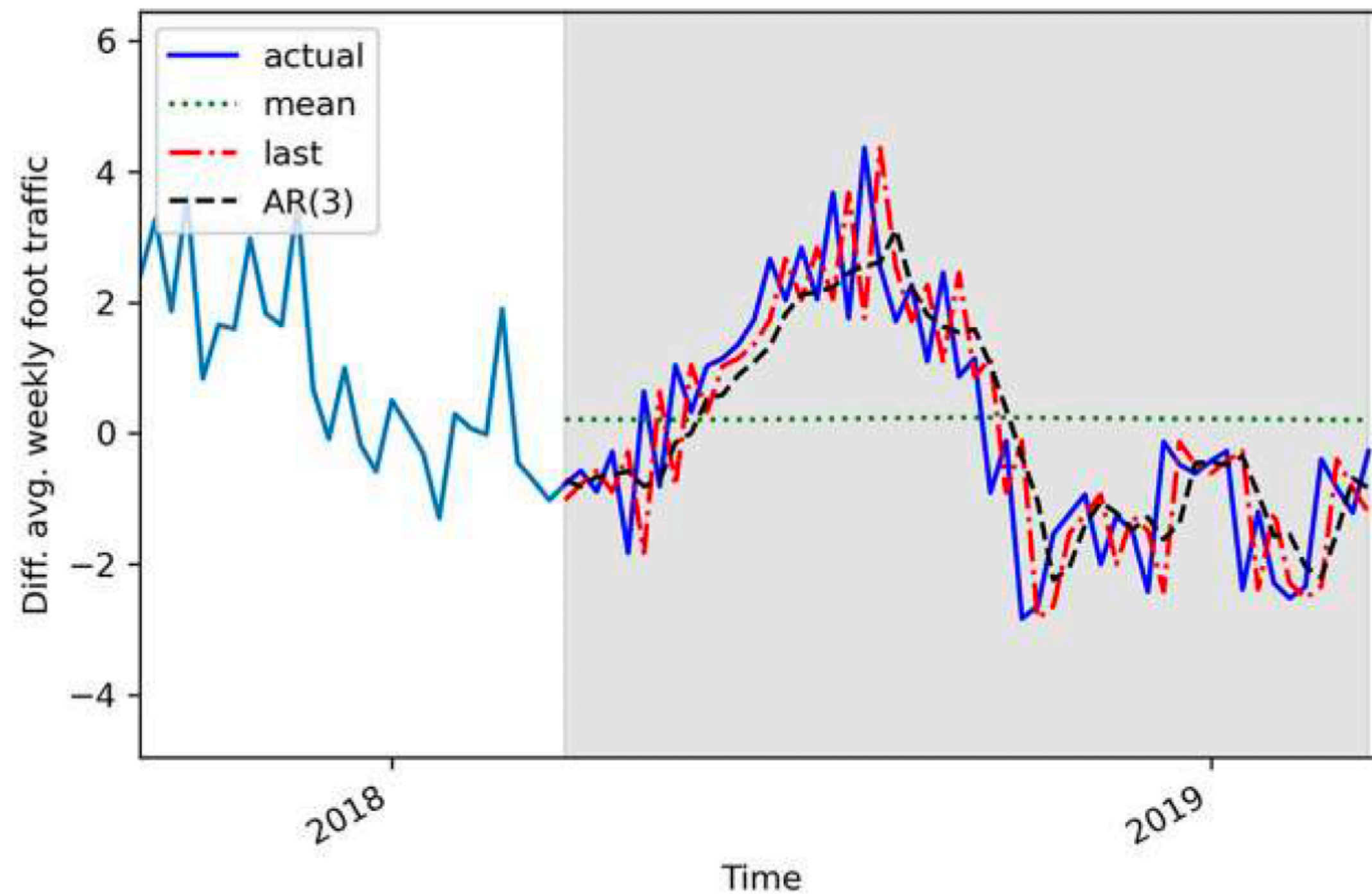
```
pred_last_value = rolling_forecast(df_diff,  
                                   TRAIN_LEN, HORIZON, WINDOW, 'last')
```

```
pred_AR = rolling_forecast(df_diff,  
                           TRAIN_LEN, HORIZON, WINDOW, 'AR')
```

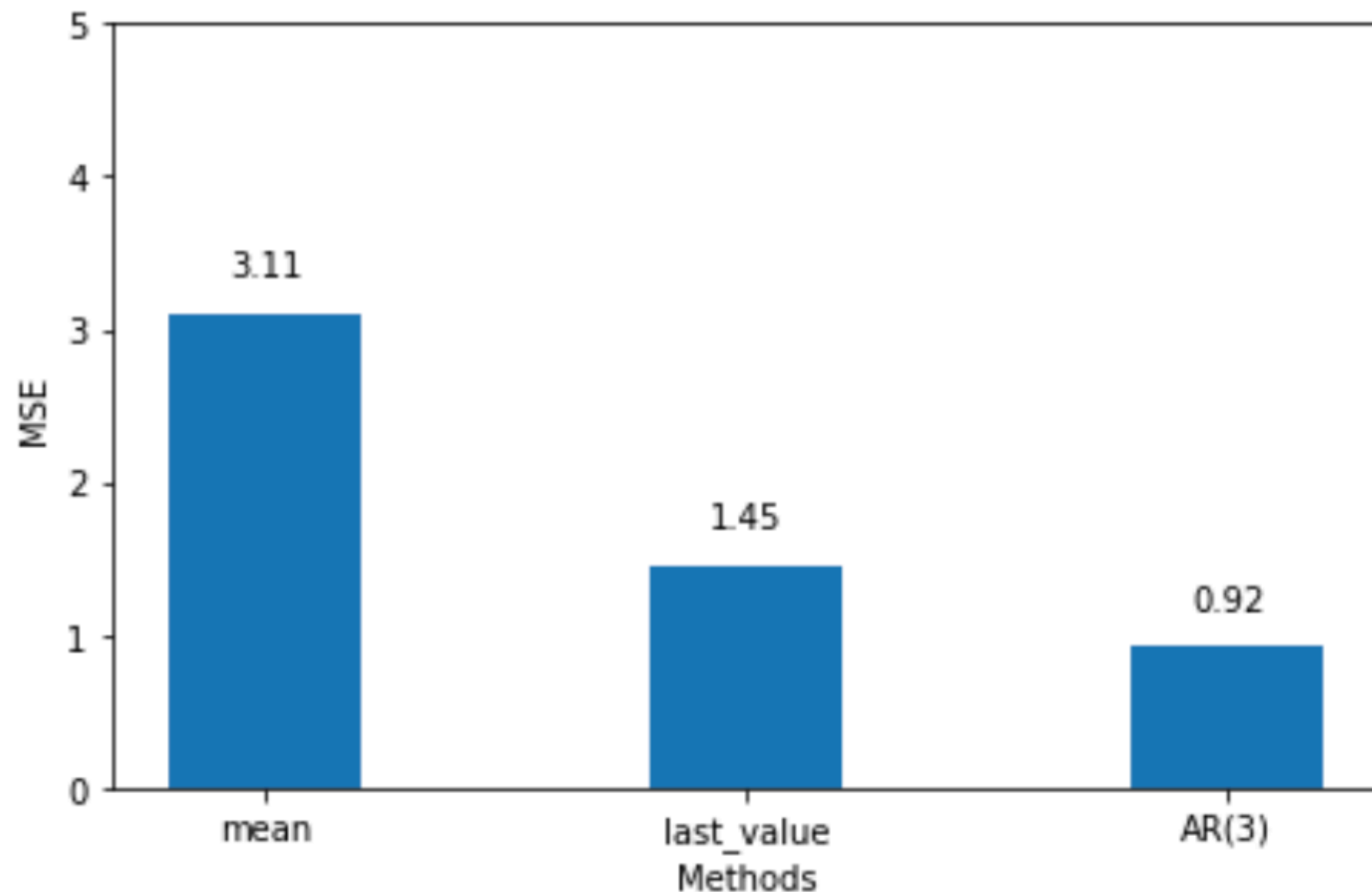
```
pred_df['pred_mean'] = pred_mean
```

```
pred_df['pred_last_value'] = pred_last_value
```

```
pred_df['pred_AR'] = pred_AR
```



Forecasting an Autoregressive Process (Comparison)



- Measure the MSE:

```
from sklearn.metrics import mean_squared_error
```

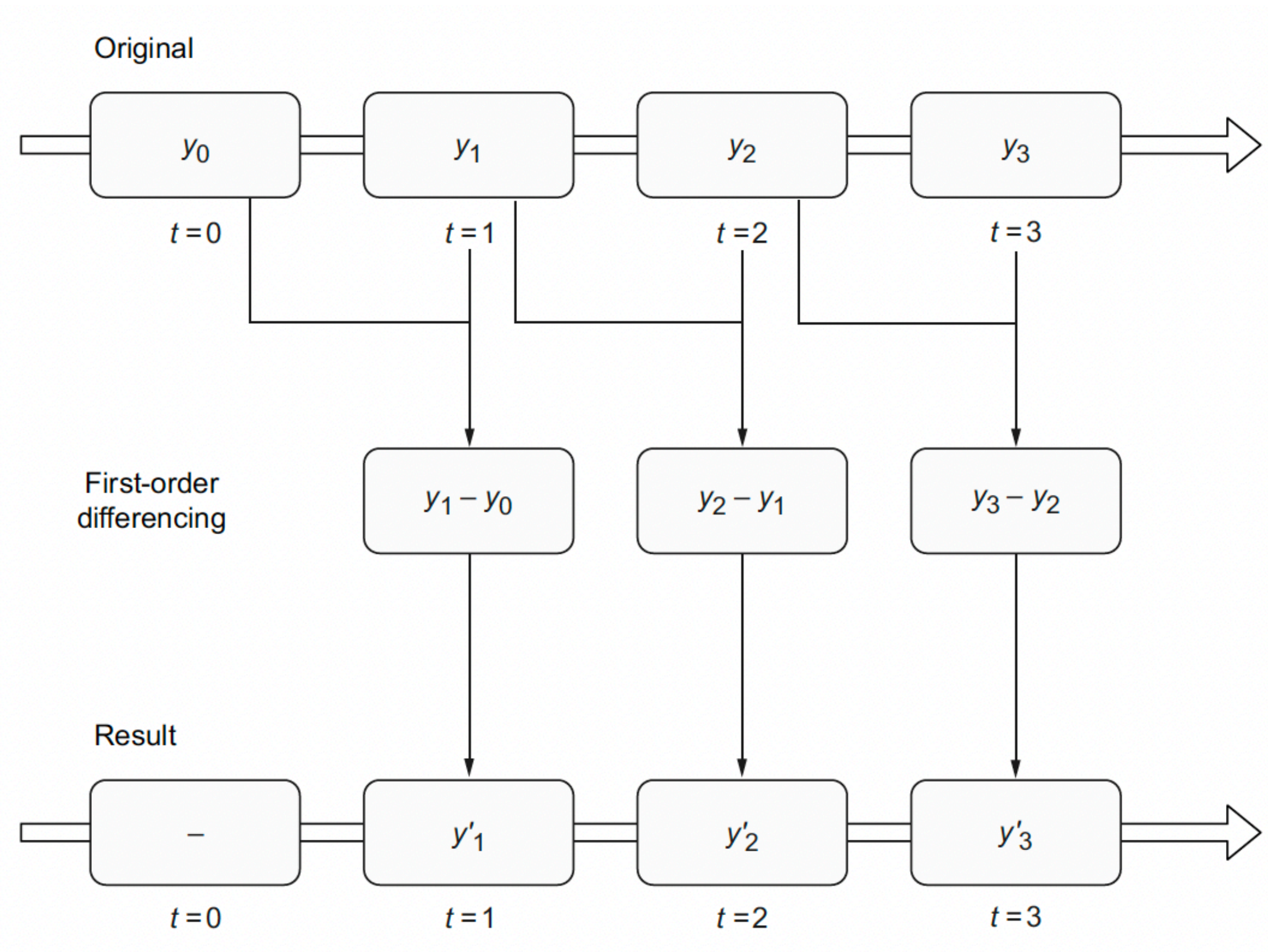
```
mse_mean =  
mean_squared_error(pred_df['foot_traffic_diff'],  
pred_df['pred_mean'])
```

```
mse_last = mean_squared_error(pred_df['foot_traffic_diff'],  
pred_df['pred_last_value'])
```

```
mse_AR = mean_squared_error(pred_df['foot_traffic_diff'],  
pred_df['pred_AR'])
```

- This gives an MSE of 3.11 for the historical mean method, 1.45 for the last value method, and 0.92 for the AR(3) model.
- This is expected, because we previously identified a third-order autoregressive process for the differenced volume of foot traffic.

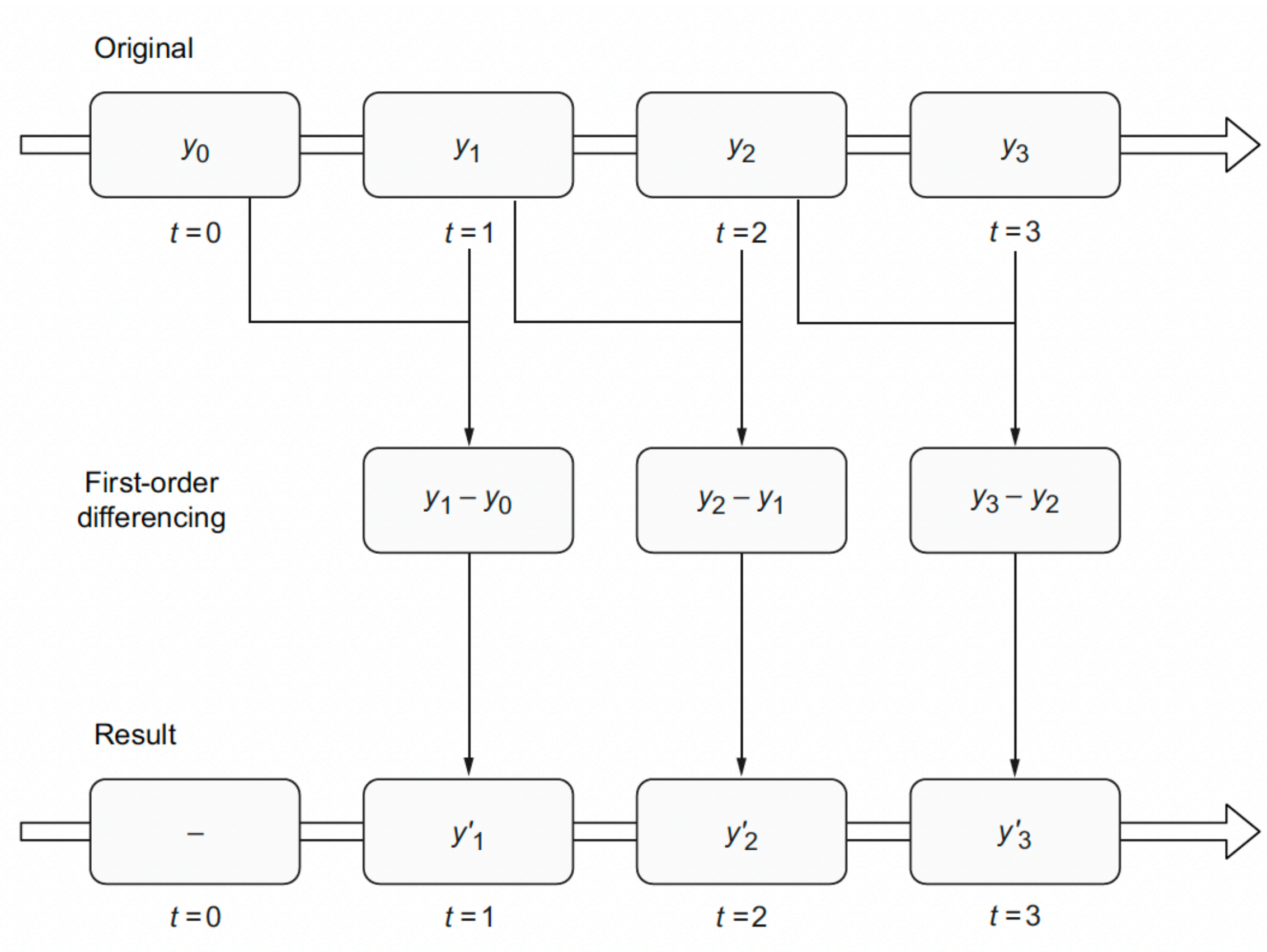
First-Order Differencing



- In order to make a time series stationary, we applied **differencing** (as many times as needed).
- **First-order differencing** calculates changes between neighbouring values:

$$y'_t = y_t - y_{t-1}$$

Reversing the First-Order Difference



- To recover the initial values from the differenced values $y'_t = y_t - y_{t-1}$, we can use the relations:

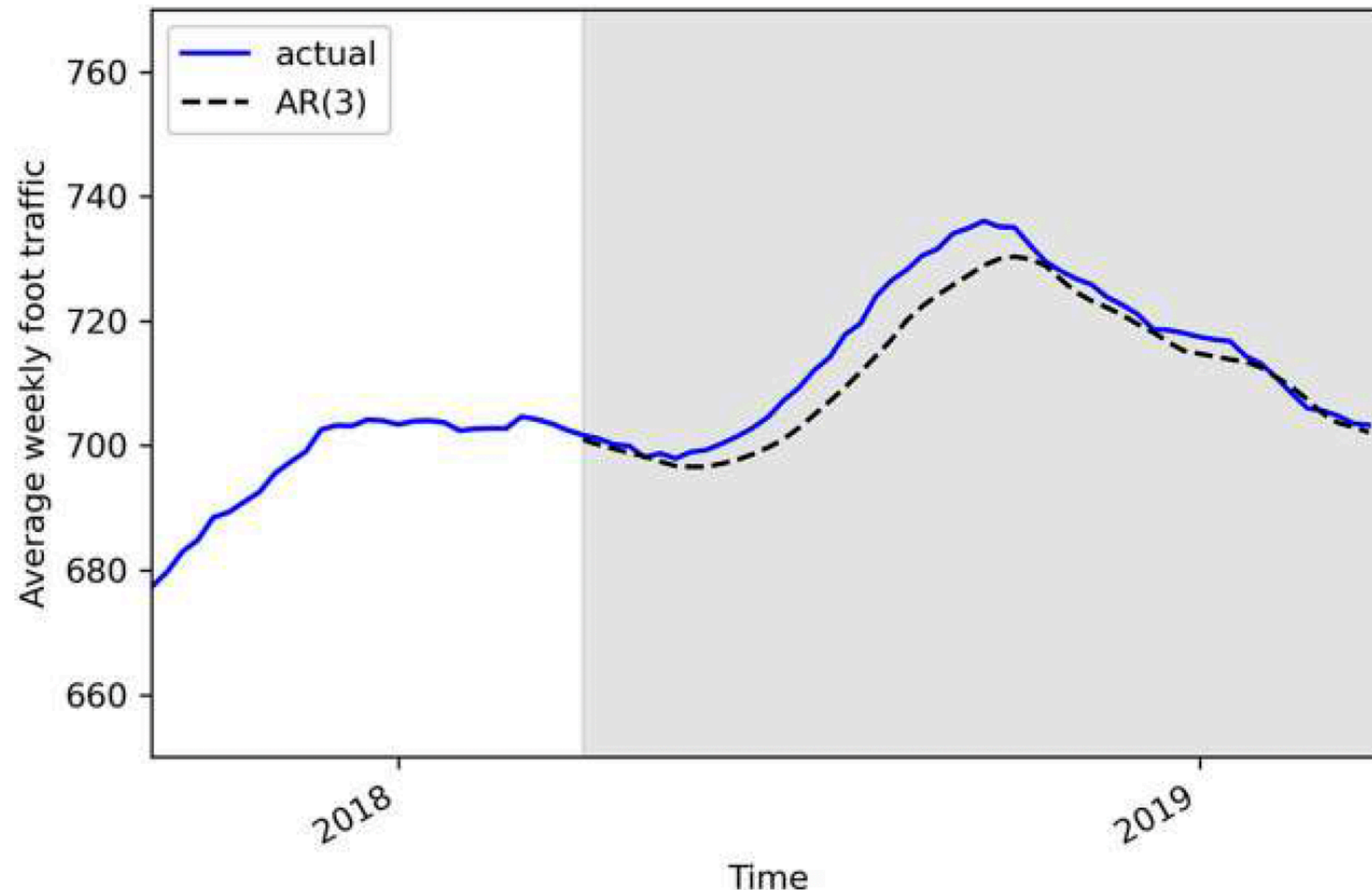
$$y_1 = y_0 + y'_1$$

$$y_2 = y_0 + y'_1 + y'_2$$

...

$$y_t = y_0 + y'_1 + y'_2 + \dots + y'_t$$

Forecasting an Autoregressive Process

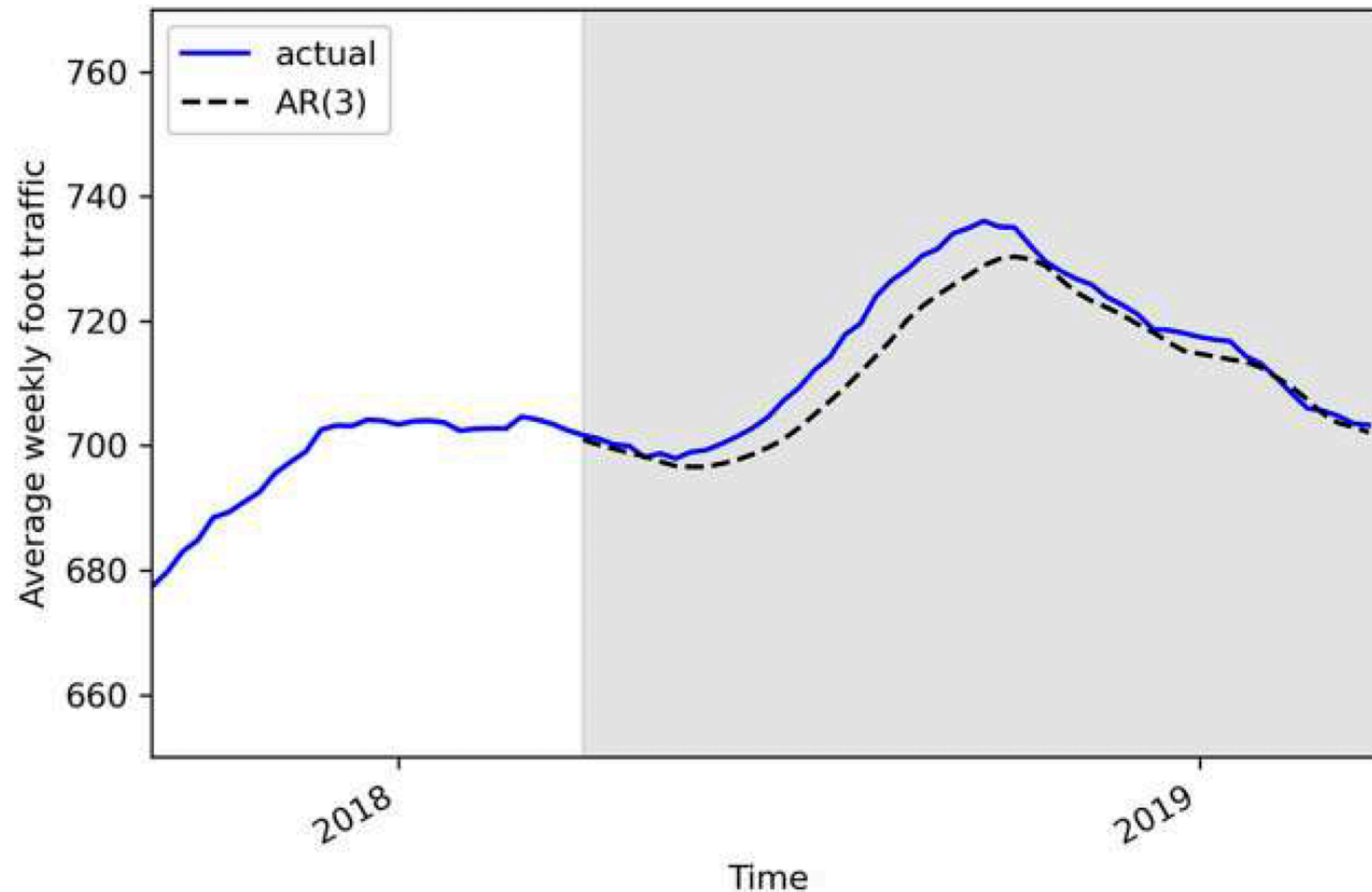


- Since our forecasts are differenced values, we need to reverse the transformation in order to bring our forecasts back to the original scale of the data; otherwise, our predictions will not make sense in a business context.
- To do this, we can take the cumulative sum of our predictions and add it to the last value of our training set in the original series. This point occurs at index 948, since we are forecasting the last 52 weeks in a dataset containing 1,000 points:

```
df['pred_foot_traffic'] = pd.Series()
```

```
df['pred_foot_traffic'][948:] =  
df['foot_traffic'].iloc[948] +  
pred_df['pred_AR'].cumsum()
```

Forecasting an Autoregressive Process



- We can measure the mean absolute error (MAE) using the **mean_absolute_error** function from **sklearn**. This metric is easy to interpret — it returns the average of the absolute difference between the predicted and actual values, instead of a squared difference like the MSE:

from sklearn.metrics import mean_absolute_error

```
mae_AR_undiff =  
mean_absolute_error(df['foot_traffic'][948:],  
df['pred_foot_traffic'][948:])
```

- This prints out a mean absolute error of 3.45. This means that our predictions are off by 3.45 people on average, either above or below the actual value for the week's foot traffic.

What is next?

- ❖ We covered the *moving average process* and how it can be modelled by an $MA(q)$ model, where q is the order. You learned that to identify a moving average process, you must study the ACF plot once it is stationary. The ACF plot will show significant peaks all the way to lag q , and the rest will not be significantly different from 0.
- ❖ We also covered the *autoregressive process* and how it can be modelled by an $AR(p)$ model, where p is the order, and it determines how many lagged values are included in the model. We also saw how plotting the ACF cannot help us determine the order of a stationary $AR(p)$ process. Instead, we must plot the PACF, in which the partial autocorrelation coefficients will be significant up until lag p only.
- ❖ However, there might be a situation where neither the ACF nor PACF gives us information. What if both the ACF and PACF plots exhibit a slow decay or a sinusoidal pattern? In that case, there is no order for the $MA(q)$ or $AR(p)$ process that can be inferred.
- ❖ This means that we are facing a more complex process that is likely a combination of both an $AR(p)$ process and an $MA(q)$ process. This is called an **autoregressive moving average (ARMA) process**, or $ARMA(p, q)$,

Thank you!